4. Hierarchical Modularity & Contracts

This section develops the modular layer of the book: how open circuits are packaged as reusable artifacts, how their boundaries are specified by interfaces and contracts, and how refinement supports hierarchical composition without losing correctness. The guiding idea is that a circuit is not merely a graph of gates, but an artifact with an exposed boundary, an internal implementation, and a measurable risk of contract violation at that boundary.

Let M denote a module with interface I, internal implementation P, and a contract C describing admissible boundary behavior. In the open-circuit setting, the interface records the arity, polarity, and typing of ports, while the contract constrains the observable relation between inputs and outputs. A module is reusable when any context that satisfies the interface assumptions can compose with the module without violating the contract. In practice, this means that the module must be parameterized by the data it depends on, rather than hard-coding assumptions that are only valid in one embedding. Parameterization is therefore not a convenience but a reusability condition: it isolates the degrees of freedom that may vary across contexts while keeping the boundary specification stable.

A useful quantitative summary of boundary reliability is the contract violation probability $\varepsilon(r)$, where r denotes the resolution level, abstraction scale, or refinement depth at which the module is inspected. At coarse resolution, $\varepsilon(r)$ measures the chance that a composed artifact fails to satisfy its declared boundary conditions when instantiated in a larger system. As refinement proceeds, one seeks a family of modules $\{M_r\}$ such that $\varepsilon(r)$ decreases, or at least remains controlled, under composition. This makes the boundary contract an executable specification rather than a purely informal promise. In the intended workflow, $\varepsilon(r)$ is not merely reported after the fact; it is tracked as part of the artifact's build and test metadata so that regressions in boundary behavior are visible at the same level as type or interface failures.

The categorical language for open circuits is provided by hypergraph categories and decorated cospans. In this view, an open circuit is represented by a cospan of interfaces into an internal wiring diagram, together with decoration data encoding the circuit structure. Hypergraph categories supply the algebraic operations needed to compose such open systems along shared boundaries, while preserving the ability to reason about ports, copying, and merging. Decorated cospans are especially well suited to modular hardware because they separate the external interface from the internal realization: two implementations with the same boundary behavior may be substituted without changing the surrounding composition. This separation is what makes the interface contract meaningful across different implementations, and it is also what allows a library of components to be reused in multiple higher-level assemblies.

This separation supports hierarchical modularity. Suppose a module M is refined into a submodule decomposition $M_1, \ldots, M_k$. A refinement map relates the abstract specification of M to the concrete implementation assembled from the M_i. The key requirement is that refinement preserve the contract: if the abstract module satisfies a property P, then the refined implementation should satisfy the corresponding instantiated property, possibly under additional assumptions made explicit in the interface. Hierarchical invariants are those properties that remain stable under repeated refinement and recomposition. Examples include arity preservation, type compatibility, monotonicity of a safety condition, and bounded error accumulation across levels. In a well-formed hierarchy, these invariants compose: if each refinement step preserves the relevant boundary data, then the entire tower of refinements remains contract-compatible with the original specification.

The compositional principle can be stated informally as follows: if each component module satisfies its local contract, and if the interfaces are matched correctly, then the assembled system satisfies the global contract up to the explicitly tracked boundary error. This is the mechanism by

which local proofs scale to larger designs. It also clarifies the role of parameterization: a module should expose precisely the parameters that affect its contract, so that refinement can substitute implementations without changing the external proof obligations. When a refinement introduces new internal structure, the additional detail should be invisible at the boundary except insofar as it improves the estimate of $\varepsilon(r)$ or strengthens the available invariants.

A concise way to package these ideas is to treat a module as a triple (I, P, C) together with a refinement witness $M' \preceq M$ indicating that the concrete artifact M' implements the abstract contract of M. Composition then becomes a boundary-matching operation on interfaces, while refinement becomes a proof obligation that the induced boundary map preserves the contract. In this language, hierarchical modularity is not merely tree-shaped decomposition; it is a disciplined propagation of interface data, contracts, and witnesses through successive levels of abstraction.

From an engineering perspective, the artifact corresponding to this section is a contract-checked library with continuous integration tests. Such a library packages modules together with interface declarations, refinement witnesses, and executable checks that validate boundary conditions on representative inputs. The CI pipeline should verify at least three layers of correctness: syntactic well-formedness of the interface, semantic preservation of the declared contract under composition, and regression tests for the refinement maps that connect abstract and concrete versions of the same module. In a mature workflow, the library becomes a reusable repository of open-circuit components whose contracts are machine-checked and whose hierarchical invariants are exercised automatically. A practical implementation may also record the measured or estimated $\varepsilon(r)$ for each release, so that the build history documents not only whether the contract holds, but how robustly it holds across refinement levels.

A concrete example clarifies the intended use of these abstractions. Consider a parameterized adder block whose interface exposes bit-width n, carry-in, and carry-out ports, while its internal implementation may be a ripple-carry, carry-lookahead, or hybrid design. The contract specifies the arithmetic relation on the boundary, together with admissible timing or error bounds. A refinement from a ripple-carry implementation to a carry-lookahead implementation changes the internal structure but should preserve the same external contract. If the library records the refinement witness and the CI suite checks the boundary relation for representative widths, then the artifact can be reused safely in larger arithmetic systems without re-proving the interface properties from scratch. This is the practical meaning of hierarchical modularity: the abstract specification remains stable while the implementation evolves beneath it.

The overall lesson is that modularity in open circuits is not only a matter of code organization. It is a mathematical discipline in which interfaces, contracts, and refinement maps jointly determine whether a component can be safely reused. Hypergraph categories and decorated cospans provide the compositional semantics; $\varepsilon(r)$ provides a boundary-sensitive error metric; and contract-checked artifacts provide the executable evidence that the hierarchy is stable under assembly and refinement. This section therefore sets up the later discussion of sequential composition and feedback by establishing the boundary discipline that makes larger compositions meaningful in the first place.